

RFC: Identifying and Opening Encrypted HDF5 File

John Mainzer john.mainzer@lifeboat.llc

WORK in PROGRESS

This RFC proposes signature of encrypted HDF5 file and its automatic detection on file opening. The document will be part of Encryption VFD design document.

1 Encrypted HDF5 File First Header Page Format

The Metadata File Header must be located at offset 0 in the metadata file, and has the following format:

Metadata File Header:

byte	byte	byte	byte
Signature			
Three Bytes Reserved for Future Use			Version
Configuration String Length			
Configuration String			
Checksum			

The fields of the first encrypted HDF5 file first header page are described in the following table.

Field Name:	Description:
Signature	Sequence of numbers intended to allow easy identification of encrypted HDF5 files. The signature shown below is a minor variation of the HDF5 file signature shown in the HDF5 file format document. Unlike the HDF5 file signature, it must always start at offset zero in the file. (Note: this will change if we decide to allow un-encrypted user blocks at the beginning of encrypted

	HDF5 files.) Decimal 137 69 72 53 13 10 26 10 Hexadecimal 89 45 48 35 0d 0a 1a 0a ASCI C \211 E H 5 \r \n \032 \n Notation This is a first cut and should be reviewed before we settle on it.
Version	One byte field containing the version of first encrypted HDF5 file header page. For now, this field will always contain zero.
Configuration String Length	Four byte field containing the length in bytes of the VFD Configuration string used to access this field. Note that this length includes the string’s terminal ‘\0’.
Configuration String	VFD stack configuration string specifying the page buffer and encrypting VFD configurations specified when creating the encrypted HDF5 file. This data is expressed as a valid VFD configuration string, specifying a VFD stack consisting of the page buffer, the encrypting VFD, and default VFD with default configuration at the bottom to perform actual file I/O. With the appropriate password, this data should be sufficient to open the encrypted HDF5 file,.
Checksum	Checksum of the contents of the header page from the beginning of the Signature through the end of the Configuration String.

2 Identification and Opening of Encrypted HDF5 Files

HDF5 files are relatively easy to identify, since they have a common eight byte signature in the first eight bytes of the super block. I say relatively, since the super block need not start at offset zero in the file. It may also start at offset 512, and other offsets at higher powers of two.

Obviously, if the logical HDF5 file is encrypted, the regular HDF5 file signature can not be used to identify it unless it is opened with the correct VFD stack – hence the need for a new signature particular to encrypted HDF5 files. This signature and associated data in the encrypted HDF5 file first header page (see format above) is intended to serve several purposes:

- Identification of encrypted HDF5 files.

- Allow generation of useful error messages when the HDF5 library attempts to open an encrypted HDF5 file with an incorrectly configured VFD stack.
- Allow extraction of the configuration string from the encrypted HDF5 file first header page. If the default VFD allows opening of an unencrypted HDF5 file, the VFD stack specified in this string should allow access to the encrypted HDF5 file¹.

While helpful, the above does not address all the issues raised by encryption. For example, we should be able to mount external HDF5 files on an unencrypted or differently encrypted base file – which will usually require a different VFD stack from that used to open the base file. While the configuration string stored in the first encrypted HDF5 file header page suggests approaches to this problem, actually addressing it will wait for another day.

2.1 Current HDF5 File Identification

At present the HDF5 library checks the file signature in two public API calls – `H5Fopen()` and `H5Fis_accessible()`². In both cases³, this test is performed as follows⁴:

1. Attempt to opening the target file using the supplied FAPL.
2. If successful, search for the HDF5 file signature via a call to `H5FD_locate_signature()`.
3. If successful, proceed with file open.

Note that a successful open of the target file in 1) is done with the VFD layer only. To a first approximation, success means that the OS calls to open the file succeeded. In the context of encrypted HDF5 files, success in opening the target file at the VFD layer may mean that the FAPL was correctly configured for an encrypted HDF5 file, or it may simply indicate the equivalent of a successful POSIX `open()` call.

In contrast, success in the second phase means that `H5FD_locate_signature()` was able to locate the expected HDF5 file signature – indicating that the target files is almost certainly an HDF5 file.

2.2 Identifying and Opening Encrypted HDF5 Files

For opening and identifying encrypted HDF5 files with appropriate error messages, we need to insert new code between opening the file with the VFD layer, and checking for the regular HDF5 signature. However, before outlining the actions that must be performed, the following case analysis may be useful.

¹ Extracting this VFD configuration string will require a new API call – tentatively named `H5Fis_encrypted()` and outlined below.

² `H5Fis_hdf5()` also checks the signature. The function is deprecated, so it will not be addressed further.

³ See `H5F_open()` and `H5F__super_read()` for `H5Fopen()`, and `H5F__is_hdf5()` for `H5Fis_accessible()`.

⁴ The control flow is a bit more complex in the case of `H5Fopen()`, but the effect is essentially the same.

If the attempt to open the file with the VFD layer fails, observe that we have nothing to do, as the VFD code (including the encrypting and page buffer VFDs if they were specified in the FAPL), will be responsible for generating any appropriate error messages.

If, on the other hand, the file open with the VFD layer succeeds, one of two things have happened:

1. The file has been opened with a VFD stack that does not include the page buffer and the encryption VFDs.
 - If the HDF5 file is encrypted, the encrypted file signature will be readable, and the regular HDF5 file signature will be encrypted and thus not readable.
 - If the HDF5 file is not encrypted, it will not have the encrypted HDF5 file signature, while the regular HDF5 file signature should appear at one of the designated offsets.
2. The file is encrypted, and the FAPL contains correct configuration data for a VFD stack that can open and read the encrypted file.

In this case, the encryption VFD will have read the encrypted HDF5 file first header page, verified encrypted HDF5 file signature, and verified that it can read the encrypted file.

Our encrypted HDF5 file signature validation code must handle all these cases gracefully.

2.3 Encrypted HDF5 File Signature Validation Outline

With the above case analysis in hand, we can outline the required encrypted HDF5 file signature validation as follows. Keep in mind that none of this happens unless the file open with the VFD layer succeeds.

1. Test to see if the VFD stack contains the page buffer and encryption VFDs.

If it does, the target file is an encrypted HDF5 file. Since the encryption VFD will have validated the encrypted HDF5 file signature, and verified that it is correctly configured to access the file, we are done.

2. If the above test failed, the VFD stack is not configured for an encrypted HDF5 file. Read the first eight bytes of the file, and test to see if the encrypted HDF5 file signature appears.

If it does, generate an error message indicating that the HDF5 file is encrypted, and suggesting that the user use `H5Fis_encrypted()` to extract the configuration string from the encrypted HDF5 file first header page⁵. This string could be loaded into the FAPL, and used to try to open the file – which should succeed if the default VFD can open an un-encrypted file on the host system.

If it does not, the file is not an encrypted HDF5 file, and we allow processing to proceed as before.

⁵ We may also implement a utility to extract the configuration string from the first header page of an encrypted HDF5 file and either store it in a file, or load the configuration into the `HDF5_DRIVER` and `HDF5_DRIVER_CONFIG` environment variables.

While attempting to load the encrypted HDF5 file signature should require no further discussion, querying for the existence and health of the page buffer and encryption VFDs does.

While there are several ways in which this could be done, the easiest is probably the `H5FD_ctl()` function defined in `H5FD.c`. Note that a new op code will be needed (see `H5FDpublic.h`), that the call to `H5FD_ctl()` will have to be made with the `H5FD_CTL_ROUTE_TO_TERMINAL_VFD_FLAG`, and that `H5FD__crypt_ctl()` will have to be updated to respond to the new op code. Finally, this new use of the `ctl` call will be yet another reason why all non-terminal VFDs must support the `ctl` callback.

A final issue is where the encrypted HDF5 file signature code should reside. While this is very much an open question, `H5FD_locate_signature()` in `H5FDint.c` should be considered.

2.4 Extracting the VFD Stack Configuration String from Encrypted HDF5 Files

Having discussed the generation of appropriate error messages in `H5Fis_accessible()` and `H5Fopen()` when an incorrectly configured VFD stack is passed to either of these functions, it is now time to consider identifying encrypted HDF5 files and extracting the VFD stack configuration string from their first header pages⁶. As suggested above, we propose addressing this issue with the new `H5Fis_encrypted()` API call, whose signature is given below:

```
herr_t
H5Fis_encrypted(bool *is_encrypted, char * file_name, HID_t fapl_id,
               char * config_str, size_t *config_str_len);
```

Processing in `H5Fis_encrypted` should proceed as follows:

1. Attempt to open the supplied file with the supplied FAPL using `H5FD_open()`. If this fails, return failure.
2. Verify that the VFD stack doesn't contain the page buffer and encrypting VFDs. If it does, the supplied stack is correctly configured to read the encrypted file, and thus the first header page will be hidden from us. Generate an appropriate error message and return failure.
3. Read the first 8 bytes of the target file, and determine if they match the encrypted HDF5 file signature. If they don't set `*is_encrypted` to false and return success.
4. Read the version of the encrypted HDF5 file first header page version. If it is not zero return failure.
5. Read the length of the configuration string. Allocate a buffer large enough for the first header page, load its contents into the buffer and verify the checksum.

⁶ Note that this problem is a general one – we have similar issues with onion files, and will have similar issues with logical HDF5 files stored on object stores and likely with files created by VOLs as well. For now, we address only the problem at hand.

6. Save a local copy of `*config_str_len` and then write the configuration string length to `*config_str_len`.
7. If `config_str` is not NULL, and the local copy of `*config_str_len` is greater than or equal to the configuration string length obtained from the encrypted HDF5 file first header page, copy the configuration string into `*config_str`.
8. Close the file, discard the buffer and return success.

Observe that `H5Fis_encrypted()` will typically be called twice – once to determine the size of the required buffer, and a second time to load the configuration string into `config_str`. Note also that the `fapl_id` will typically be `H5P_DEFAULT`.

The configuration string can be loaded into a FAPL via the public version of `H5CL_load_vfd_config_str_into_fapl()` and then used in a subsequent call to `H5Fopen()`.

2.5 Routing the Configuration String to the Encrypting VFD

A final question is how does the encrypting VFD obtain the page buffer configuration? Recall that this information must be error checked and stored in the first encrypted HDF5 file header page.

Three solutions come to mind:

1. After the page buffer successfully constructs the underlying VFD stack, use the `H5FD_ctl()` call to pass the page buffer configuration data down the stack to the encrypting VFD. Best do this in binary form since constructing the desired configuration string in the encrypting VFD will be easier than filtering any intervening VFDs out of the configuration string. This approach delays setup of the first encrypted HDF5 file header page, but uses existing infrastructure.
2. Create a new, private FAPL message, and use this message to pass the page buffer configuration (again in binary form) down to the underlying VFD when setting up the underlying VFD stack. This avoids the delay in setting up the first encrypted HDF5 file header page, but will break if there is an intervening VFD between the page buffer and the encrypting VFD.
3. Generalize the first solution by broadcasting the address of the top level VFD in the stack to all VFDs in the stack. This would allow any VFD to use the `ctl` call to talk to VFDs higher in the stack. For now at least, this has all the problems as the first solution with greater overhead. That said, if this problem arises regularly, it may be worth doing.

Unless we come up with something better, I lean towards the first approach.

2.6 Automatic VFD Stack Configuration

As should be obvious, the new `H5Fis_encrypted()` call points a way to automatic VFD stack reconfiguration. We don't address this now for two reasons:

The obvious reason is that we don't need it for the initial release of encryption support, and implementing it cleanly will require a fair bit of thought and review of the HDF5 library code.

The more fundamental reason is that problem of opening an HDF5 file that can't be opened with the sec2 driver where ever the sec2 driver is applicable, is a recurrent one, that should have a general solution.

We are addressing the encryption special case of this problem with a new signature and VFD stack configuration string in a header page. If this approach works out, it should be extensible to similar cases – onion files, files created by VOL stacks, logical files managed as pages on object stores, collections of sub-files, etc. Viewed this way, a logical next step would be a more generic version of H5Fis_encrypted() followed by an equally generic solution to automatic VFD stack configuration.